

SYNOPSIS

SYNOPSIS

TITLE: SMART COMMUNITY SERVICES

Aim:

The aim of this project is to deliver a transparent, unified multi-society digital ecosystem for gated community residents, security guards, domestic staff, and administrators to streamline property operations, multi-estate management, visitor security, helper ratings and attendance, maintenance billing, helpdesk ticketing, and real-time emergency safety on a single platform.

Description:

Project Overview: Multi-service enterprise platform combining multi-society gated community infrastructure management with integrated resident lifestyle services (pre-approved gate passes, parcel OTP verification, staff attendance tracking & star ratings, maintenance ERP billing, helpdesk ticketing, community noticeboards, full administrative CRUD control, and real-time SOS safety radar) tailored for modern gated communities.

Core Portals:

- **Resident Portal:** Pre-approved visitor gate pass generation, parcel arrival OTP viewing, domestic staff directory with 1-to-5 star rating & review submission, maintenance bill payment, helpdesk ticketing with attachment uploads, and community broadcasts.
- **Admin Command Centre:** Multi-society hierarchy management (societies, blocks, flats, residents, security guards, domestic staff), full-CRUD edit and delete controls across all registries, automated maintenance invoice generation, ticket SLA tracking, and audit logging.
- **Security Guard Console:** Society-scoped gate pass passcode/QR verification, parcel inward & OTP collection, daily helper attendance punching, and live SOS emergency radar with high-decibel audio/visual alerts.

Modules:

- **User Authentication & Profile Module:** Handles secure multi-role signup, login, JWT session management, and role-based access control (RBAC) for Admins, Residents, and Security Guards.
- **Society Infrastructure & Multi-Estate Module:** Supports multiple residential societies (Prestige Tranquility, Lake View, Lake View Apartments), block definitions, and flat registry management.

SMART COMMUNITY SERVICES

- **Visitor Security & Gate Pass Module:** Enables residents to create pre-approved visitor gate passes with 6-digit PIN codes / QR passes, allowing security guards to verify credentials at entry gates and log entry/exit timestamps.
- **Parcel & Delivery Management Module:** Allows gate guards to log incoming packages, automatically generating 4-digit collection OTPs and firing real-time WebSockets notifications.
- **Domestic Staff Directory & Ratings Module:** Maintains a multi-category directory of daily helpers (maids, cooks, drivers, plumbers, electricians), enabling guard attendance punching, flat assignments, and resident 1-to-5 star review submissions.
- **Admin Full-CRUD Master Management Module:** Grants administrators full creation, edit (update), and deletion capabilities across all 6 core registries (Society Master, Blocks, Flats, Residents, Security Guards, Domestic Staff).
- **ERP Billing & Maintenance Ledger Module:** Automates monthly maintenance invoice generation, tracks charge breakdowns, supports online payments, and maintains flat ledgers.
- **Helpdesk Ticketing & SLA Module:** Enables residents to raise maintenance tickets with photo uploads, track status changes (Open, In Progress, Resolved), and assign tickets to staff.
- **Community Noticeboard & Real-Time SOS Radar Module:** Facilitates community broadcasts, amenity bookings, and features a zero-latency Socket.io emergency SOS radar alerting guards instantly during medical or fire emergencies.

INTRODUCTION

INTRODUCTION

Introduction

The modern urban gated community landscape remains heavily fragmented, forcing residents, society management committees, and security guards to rely on disconnected tools for visitor security, helper attendance, maintenance billing, and emergency responses. This platform addresses these inefficiencies by unifying the entire multi-society lifecycle into a single digital solution. Built on a modern decoupled architecture, the backend utilizes Node.js and Express.js with TypeScript to execute robust business logic, Prisma ORM for type-safe SQLite database operations, and Socket.io for bi-directional, real-time emergency events. The frontend leverages React 18 with Vite and Tailwind CSS to deliver an ultra-responsive, accessible user interface across desktop and mobile devices.

Overview:

User-Side Activity (Resident Portal)

- **Visitor Pass & Delivery Control:** Generate pre-approved 6-digit visitor passes for guests and view parcel arrival OTPs in real-time.
- **Domestic Staff & Star Ratings:** Browse society daily helpers (maids, cooks, drivers, plumbers), check 1-5 star ratings, submit reviews, and receive gate entry notifications.
- **Maintenance & Billing:** View monthly society maintenance invoices, inspect itemized charge breakdowns, and pay bills securely online.
- **Helpdesk & Ticketing:** Submit priority tickets with photo attachments and receive live resolution updates from society managers.

Admin-Side Activity (Command Centre)

- **Infrastructure & Full-CRUD Management:** Add, edit, and delete records across Society Master Registry, Blocks Management, Flats Registry, Residents Directory, Security Guards Directory, and Domestic Staff Directory.
- **Multi-Society Scoping:** Filter and allocate resources across multiple residential estates (Prestige Tranquility, Lake View, Lake View Apartments).

SMART COMMUNITY SERVICES

- **ERP Billing Operations:** Batch-generate monthly maintenance invoices, log manual payment updates, and review financial ledgers.
- **Helpdesk Operations:** Monitor open resident tickets, assign technicians, and track SLA resolution metrics.
- **Community Notices:** Publish society broadcasts, emergency alerts, and amenity facility guidelines.

Guard-Side Activity (Security Console)

- **Gate Verification:** Verify 6-digit passcodes for pre-approved visitors and log vehicle numbers at gate checkpoints.
- **Parcel Inward Logging:** Log incoming courier packages and generate secure 4-digit collection OTPs.
- **Helper Attendance Punching:** Punch check-in and check-out times for domestic maids, drivers, and maintenance staff.
- **Live SOS Radar:** Monitor high-priority emergency alerts triggered by residents with instant audio-visual alarms.

Purpose:

The core purpose of this platform is to eliminate operational friction and safety vulnerabilities in gated residential communities by consolidating visitor entry, staff management, financial accounting, maintenance servicing, and emergency dispatching into a unified digital workspace. By bridging residents directly with gate guards and administrative management, the system eliminates paper registers, prevents unauthorized entry, speeds up maintenance resolution, and guarantees instant emergency assistance. Ultimately, the platform aims to elevate urban living standards through transparent automation and real-time connectivity.

OBJECTIVE

OBJECTIVE

The main objective of this project is to build an integrated, full-stack enterprise platform that unifies gated society administration with comprehensive resident security and living solutions. It seeks to replace manual gate logs, paper billing, and delayed complaint handling with a single digital ecosystem featuring transparent financial accounting, verified visitor authentication, automated attendance tracking, and zero-latency emergency SOS signaling. Ultimately, the platform aims to set a high benchmark for modern smart community platforms.

Gate & Visitor Security Objective

The visitor security engine is designed to eliminate perimeter vulnerabilities across residential gated societies. By replacing physical logbooks at security gates with a digital pass verification system, residents can generate 6-digit passcodes or QR passes prior to a guest's arrival. Security guards can instantly verify these credentials on the Guard Console, recording entry/exit timestamps, vehicle numbers, and guard IDs into an immutable audit trail. Furthermore, the parcel management module ensures seamless courier handoffs by sending 4-digit collection OTPs directly to residents, preventing package loss and unauthorized pickups.

Financial ERP & Invoicing Objective

The financial accounting objective establishes an automated billing engine that streamlines society maintenance collection and ledger management. Administrators can batch-generate itemized monthly invoices based on flat square footage or fixed rates, detailing maintenance fees, sinking fund allocations, water usage, and common area upkeep. Residents receive instant billing alerts, view breakdown summaries, and complete online payments, while the system automatically updates payment states (Pending, Paid, Overdue) and maintains immutable financial ledgers for complete auditability.

Helpdesk & Maintenance Objective

The helpdesk objective focuses on structured issue resolution for society infrastructure and resident flats. Residents can raise tickets categorized under plumbing, electrical, carpentry, security, or common area maintenance, attaching photographic evidence directly from their devices. The administrative command center tracks ticket lifecycle states—Open, In Progress, Resolved—allowing managers to assign technical staff, post progress updates, and monitor resolution timelines to enforce strict service level agreements (SLAs).

Real-Time Safety & SOS Radar Objective

The emergency objective prioritizes life safety by implementing a zero-latency WebSockets communication radar. Residents facing medical emergencies, fire hazards, or security threats can trigger an instant SOS alert with a single tap. The system immediately broadcasts the alert to all active security guard consoles, activating visual radar animations and high-decibel audio alarms while pinpointing the exact flat number, block, and resident contact details for rapid emergency dispatch.

User Experience (UX) Objective

The user experience objective focuses on delivering an ultra-responsive, accessible interface featuring native Dark and Light modes built with Tailwind CSS design tokens. Incorporating high-contrast visual cues, sleek backdrop blurs, clean typography, and intuitive navigation grids, the UI ensures seamless usability for residents, elderly users, and security guards alike. Asynchronous REST APIs paired with Socket.io push events guarantee live data synchronization without page reloads across smartphones, tablets, and desktop computers.

SOFTWARE AND HARDWARE REQUIREMENTS

SOFTWARE AND HARDWARE REQUIREMENTS

1. Hardware Requirements:

The hardware system specification consists of:

Hardware Component	Minimum System Specification
Processor	AMD Ryzen 5 5625U / Intel Core i5 (2.3 GHz or higher)
Operating System	Windows 11 Pro / Linux Ubuntu 22.04 LTS / macOS
Hard Disk	512 GB NVMe SSD
Monitor	15" Full HD LED Display (1920 x 1080 resolution)
Input Devices	Standard Keyboard, Optical Mouse / Touchscreen
RAM	16 GB DDR4 / DDR5

2. Software requirements:

The software stack used to develop and run the application consists of:

Software Tool	Selected Technology Stack
Operating System	Windows 11 / Linux Ubuntu 22.04 LTS
Programming Language	TypeScript (v5.0+), JavaScript (ES6+ Node.js v20+)
Web Framework	Express.js (Server Engine), React 18 with Vite (Frontend)
Database & ORM	SQLite (`dev.db`), Prisma ORM (v5.0+)
Development IDE	Visual Studio Code (VS Code v1.85+)
Documentation	Microsoft Word, HTML5 / CSS3 Print Engine, Markdown

Software Environment

- **Operating System:** Supports cross-platform deployment across Linux (Ubuntu 22.04 LTS recommended for production server stability), Windows 11, or macOS (Sonoma/Sequoia), ensuring consistent system dependency handling and environment parity across development and production environments.
- **Runtime & Backend Framework:** Utilizes Node.js v20+ as the runtime execution engine alongside Express.js with TypeScript, providing strong compile-time type safety, structured modular routes, non-blocking asynchronous I/O event loops, and optimal memory utilization for concurrent API requests.
- **ORM & Database Engine:** Employs Prisma ORM for type-safe database access, automated schema migrations, and relational modeling over an embedded SQLite engine (`dev.db`), guaranteeing full ACID compliance and lightweight portable data management.
- **Frontend Engine & UI Framework:** Built using React 18 and Vite for lightning-fast client-side module bundling and virtual DOM rendering, styled using Tailwind CSS custom color tokens for high-contrast dark/light themes, and enhanced with Lucide React vector iconography.
- **Real-Time Communication:** Leverages Socket.io client and server WebSockets engine for instant bi-directional signaling, powering zero-latency emergency SOS alerts, instant gate pass check-in notifications, and parcel arrival pings.

FRONT END AND BACK END

FRONTEND

The web front-end application is developed using modern web technologies including HTML5, CSS3, JavaScript (ES6+), TypeScript, and React 18, delivering an intuitive, responsive, and accessible interface for all user roles.

React 18 Architecture

React is an open-source, component-based front-end JavaScript library maintained by Meta and a vibrant developer community. In this project, React 18 powers single-page application (SPA) navigation via React Router DOM v6, enabling seamless transitions between Admin, Resident, and Guard views without full page reloads.

TypeScript for Client Logic

TypeScript extends JavaScript by adding strict compile-time type definitions. In the frontend layer, TypeScript interfaces define clear prop contracts, API response payloads, and state models, preventing runtime undefined errors and memory leaks during complex user interactions.

Tailwind CSS Styling & Design Tokens

Tailwind CSS is a utility-first CSS framework that enables rapid UI development. Custom design tokens defined in `index.css` provide sleek dark-mode aesthetics with subtle backdrop blurs, glassmorphism card overlays, neon status indicators, and fluid CSS grid structures tailored for both mobile smartphones and widescreen desktop monitors.

Component-Based Architecture

The client side is organized into modular reusable components, separating layout containers, authentication views, interactive modal dialogs, and role-specific dashboards:

- **Layout Components:** AdminLayout, ResidentLayout, and GuardLayout provide consistent top navigation bars, sidebar links, and user session displays.
- **UI Components:** AnimatedCard, AnimatedModal, StatCard, and custom button controls deliver interactive visual feedback and fluid transitions.
- **Module Pages:** Dedicated pages for AdminDashboardPage, ResidentPortalPage, GuardConsolePage, GuardPassVerifyPage, and GuardSOSRadarPage isolate business workflows into clean, maintainable units.

State Management & Sockets

Client state is managed using React Context API (AuthContext and SocketContext). The SocketContext maintains an active WebSocket connection to the Node.js backend, listening for live event triggers such as sos_alert_triggered, visitor_pass_created, and gate_entry_alert to update the DOM instantly.

BACKEND

Introduction to Node.js & Express.js

Node.js is a cross-platform, open-source JavaScript runtime environment built on Chrome's V8 engine that executes JavaScript code outside a web browser. Released by Ryan Dahl in 2009, Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient, perfect for data-intensive real-time applications running across distributed devices.

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It simplifies request routing, middleware integration, JSON payload parsing, and HTTP response handling. In "Smart Community Services," Express.js acts as the server backbone, serving RESTful API endpoints under /api/auth, /api/visitor, /api/staff, /api/erp, /api/helpdesk, and /api/community.

Clean Architecture & TypeScript Integration

The backend codebase enforces strict separation of concerns through a modular folder structure. Each module encapsulates its data schemas, route handlers, and business logic controllers (e.g., auth.controller.ts, visitor.controller.ts, erp.controller.ts), guaranteeing high maintainability and testability.

Backend Design Highlights

- **Asynchronous Non-Blocking I/O:** Node.js handles thousands of concurrent socket connections and API requests on a single thread using its event loop, preventing server thread starvation.
- **Middleware Pipeline:** Custom Express middleware handles JWT token verification (auth.ts), role-based permission validation (roleGuard), Zod input validation, and centralized error handling (errorHandler.ts).
- **Security & Hashing:** Passwords are encrypted using bcrypt with salt rounds, ensuring secure storage in the database. API authentication relies on signed JSON Web Tokens (JWT) with 7-day expiration windows.
- **Automated Background Jobs:** Integrated node-cron scheduled tasks automate monthly maintenance bill calculations and send automated status reminders without manual administrative intervention.
- **Real-Time Socket Gateway:** Socket.io server initialization attaches to the HTTP server, managing authenticated room joins (room:guards, room:admin, flat:id) for targeted event broadcasting.

DATABASE

SQLite Database Engine

SQLite is an in-process library that implements a self-contained, serverless, zero-configuration, transactional SQL database engine. Unlike traditional databases like MySQL or PostgreSQL, SQLite stores the entire database as a single cross-platform file (dev.db) on disk, eliminating networking latency and administrative overhead while guaranteeing full ACID compliance.

Prisma ORM Integration

Prisma is a next-generation Object-Relational Mapper (ORM) for Node.js and TypeScript. It defines database tables and relations using a clean declarative file (schema.prisma) and generates a strongly-typed Prisma Client API for database operations.

Benefits of Prisma ORM:

- **Type Safety:** Automatically generates TypeScript interfaces matching database tables, eliminating SQL injection risks and type mismatches.
- **Automated Migrations:** prisma migrate handles database schema evolution cleanly without data loss.
- **Relational Mapping:** Seamlessly manages foreign keys, cascading deletions, unique constraints, and multi-table joins.

FRAMEWORKS AND LIBRARIES

Express.js Web Server Framework

Express.js provides the core HTTP processing engine for the application, mapping incoming REST endpoints to specific controller logic, parsing JSON bodies, managing CORS policies, and returning formatted JSON responses via `sendSuccess()` utility helpers.

Socket.io WebSockets Gateway

Socket.io enables real-time, bi-directional, event-based communication between client browsers and the backend server. It falls back gracefully to HTTP long-polling if WebSockets are restricted by local networks, ensuring reliable delivery of critical emergency SOS signals.

Zod Validation Library

Zod is a TypeScript-first schema validation library with static type inference. It validates incoming request payloads at the API boundary, guaranteeing that fields like passcodes, email formats, and numeric amounts satisfy strict validation rules before executing database logic.

SMART COMMUNITY SERVICES

Summary of Technology Integration

Layer	Technology	Core Responsibility
Frontend UI	React 18 + Vite + Tailwind CSS	Single-Page Application, responsive dark/light UI, interactive dashboards.
Client State	React Context API + Socket Client	User session state, JWT token storage, real-time WebSocket event listeners.
Backend API	Node.js + Express.js + TypeScript	REST API endpoints, JWT auth middleware, Zod payload validation, error handling.
Real-Time Engine	Socket.io Server Engine	Room-based WebSocket broadcasting (room:guards, flat:id), SOS emergency radar.
Database & ORM	Prisma ORM + SQLite (dev.db)	ACID relational data storage, multi-entity mapping, migration management.

SYSTEM DESIGN

SYSTEM DESIGN

ER DIAGRAM:

An Entity-Relationship (ER) Diagram is a vital tool in database design that visually represents the data model of a system. It defines entities, their attributes, and the structural relationships between entities. This diagram acts as a conceptual blueprint for the system's database schema, ensuring proper normalization and data integrity.

In an Entity-Relationship (E-R) diagram, relationships connect entities to demonstrate how data points interact. Common relationship types include:

Types of Relationships:

1. One-to-One (1:1): Each instance of Entity A is related to exactly one instance of Entity B, and vice versa.

Example: Each flat has one assigned owner account.

2. One-to-Many (1: N): Each instance of Entity A is related to one or more instances of Entity B, but each instance of Entity B relates to only one instance of Entity A.

Example: One block contains many flats.

SMART COMMUNITY SERVICES

3. Many-to-One (N:1): The inverse of a one-to-many relationship.

Example: Many visitor passes are created by one resident user.

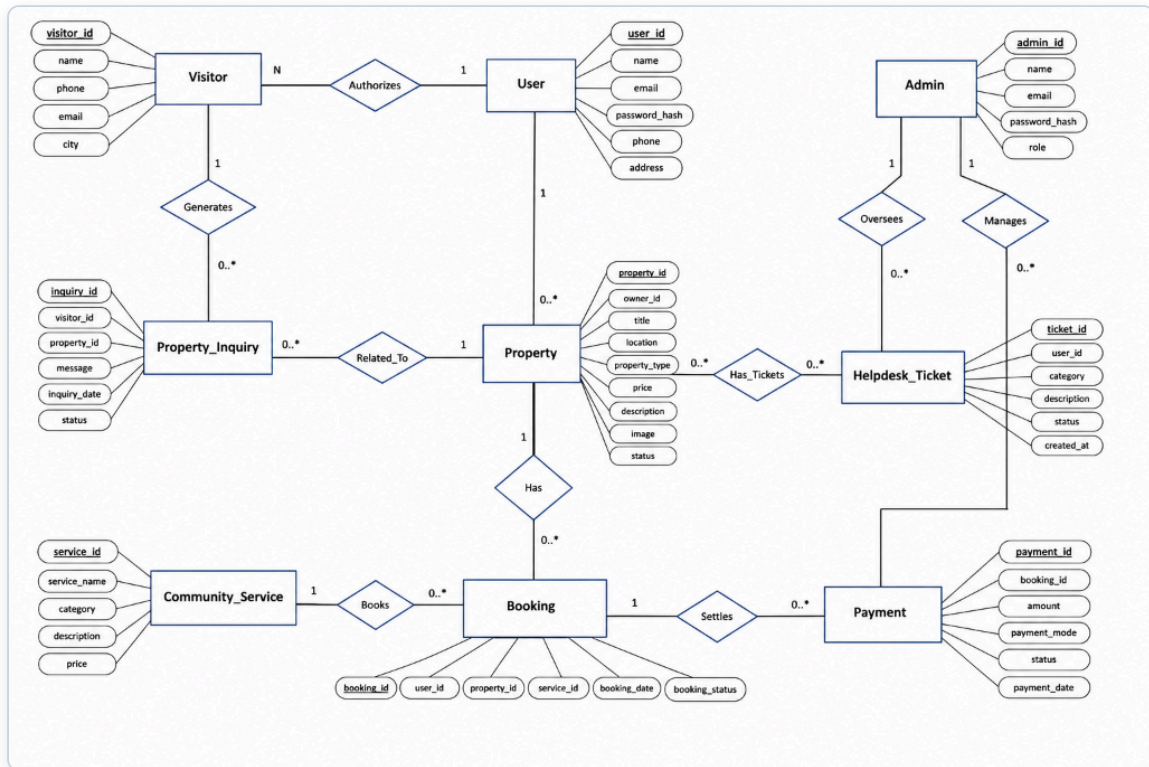
4. Many-to-Many (M: N): Instances of Entity A relate to multiple instances of Entity B, and vice versa. Requires a junction table.

Example: Staff members assigned to multiple flats via StaffAssignment.

ER Diagram Notational Symbols:

Symbol Name	Geometric Shape	Description / Purpose
Entity	Rectangle	Represents a real-world object or data table (e.g., User, Flat, Ticket).
Weak Entity	Double Rectangle	Entity dependent on a strong entity for primary identification (e.g., Comment).
Attribute	Ellipse	Represents a property or field of an entity (e.g., email, status, price).
Key Attribute	Underlined Ellipse	Represents the primary key field uniquely identifying records (e.g., <u>id</u> , <u>code</u>).
Relationship	Diamond	Represents an association between two or more entities.

ER DIAGRAM FOR SMART COMMUNITY SERVICES



The Entity-Relationship (ER) Diagram above illustrates the conceptual data model and relational mappings for the Smart Community Services platform, detailing core entities (User, Admin, Visitor, Property, Property Inquiry, Booking, Community Service, Payment, and Helpdesk Ticket) alongside their key attributes, relationships, and cardinalities.

DATA FLOW DIAGRAM:

A Data Flow Diagram (DFD) is a graphical representation of how data flows through an information system. It depicts system inputs, processing steps, data storage, and outputs without detailing physical hardware or implementation specifics.

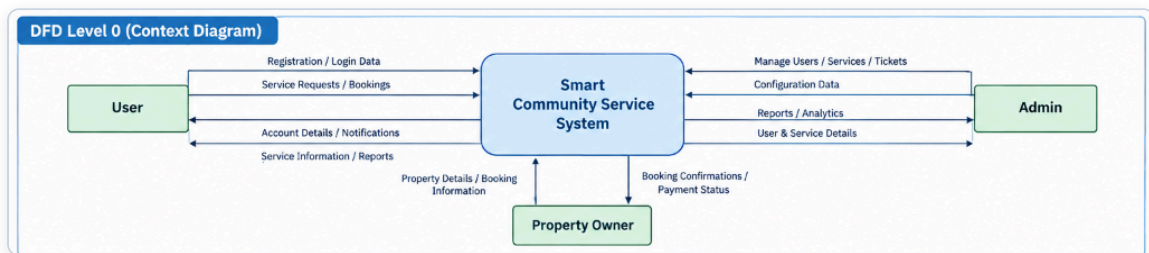
DFD Symbols & Notation:

DFD Element	Graphical Notation	Description
External Entity	Rectangle	External source or destination of data (e.g., Resident, Admin, Guard).
Process	Circle / Rounded Box	Transforms incoming data into outgoing data (e.g., Pass Verification).
Data Store	Open Parallel Lines	Repository of data at rest (e.g., D1: Users, D2: Visitor Passes).
Data Flow	Arrowed Line	Indicates the directional movement of data packages.

SMART COMMUNITY SERVICES

DATA FLOW DIAGRAM (DFD) — LEVEL 0 (CONTEXT DIAGRAM)

The Context Level DFD (Level 0) represents the high-level boundary of the Smart Community Service System, demonstrating data flows between external entities (User, Admin, Property Owner) and the central system engine.

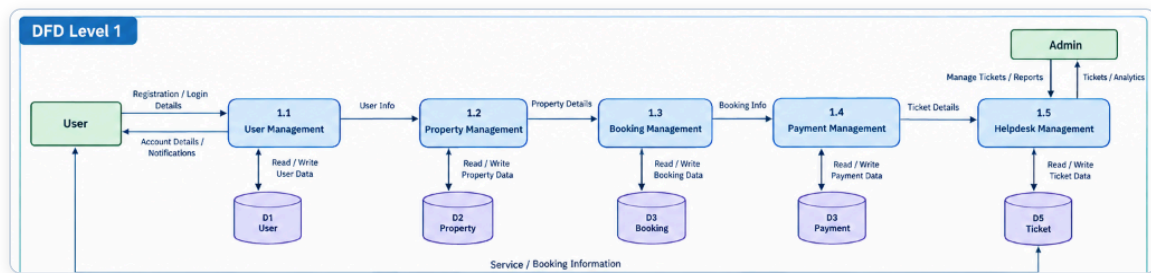


External Entity Interactions:

- **User:** Sends Registration / Login Data, Service Requests / Bookings; receives Account Details, Notifications, Service Info & Reports.
- **Admin:** Manages Users / Services / Tickets, inputs Configuration Data; receives Reports / Analytics and User & Service Details.
- **Property Owner:** Exchanges Property Details / Booking Info and receives Booking Confirmations / Payment Status.

DATA FLOW DIAGRAM (DFD) — LEVEL 1 (SYSTEM PROCESSES)

The Level 1 DFD decomposes the core system into primary process modules (User Management, Property Management, Booking Management, Payment Management, Helpdesk Management) and their respective data stores.

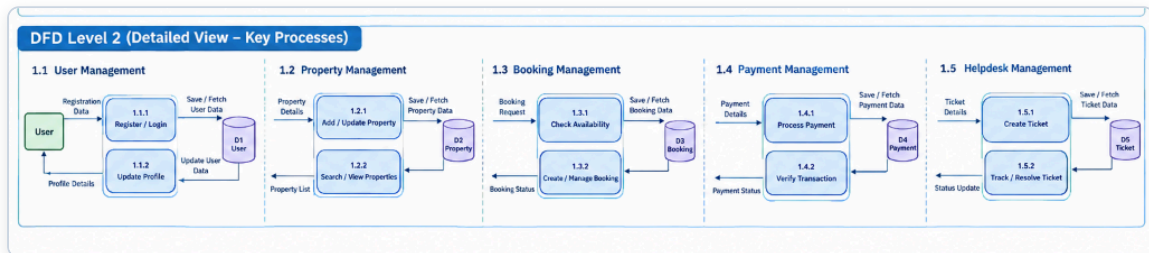


Primary Processes & Data Stores:

- **1.1 User Management:** Interacts with *D1 User* store for authentication, profile setup, and RBAC controls.
- **1.2 Property Management:** Manages property listings, unit metadata, and reads/writes to *D2 Property* store.
- **1.3 Booking Management:** Handles service/facility reservation requests and logs to *D3 Booking* store.
- **1.4 Payment Management:** Processes transactional financial settlements and updates *D3 Payment* store.
- **1.5 Helpdesk Management:** Routes resident complaint tickets to Admin and persists updates to *D5 Ticket* store.

DATA FLOW DIAGRAM (DFD) — LEVEL 2 (DETAILED KEY PROCESSES)

The Level 2 DFD provides an exploded, granular view of specific subprocesses within each major functional module.



Granular Subprocess Breakdown:

- **1.1 User Management:** 1.1.1 Register / Login & 1.1.2 Update Profile.
- **1.2 Property Management:** 1.2.1 Add / Update Property & 1.2.2 Search / View Properties.
- **1.3 Booking Management:** 1.3.1 Check Availability & 1.3.2 Create / Manage Booking.
- **1.4 Payment Management:** 1.4.1 Process Payment & 1.4.2 Verify Transaction.
- **1.5 Helpdesk Management:** 1.5.1 Create Ticket & 1.5.2 Track / Resolve Ticket.

IMPLEMENTATION

IMPLEMENTATION

Tools and Technologies Used

To implement this project, Visual Studio Code was utilized as the primary integrated development environment (IDE). Visual Studio Code is a streamlined, open-source code editor featuring integrated debugging, Git version control, TypeScript IntelliSense, and extensive extension support. The project monorepo structure contains a Node.js Express server backend and a Vite React frontend client.

Core Stack Requirements:

- **HTML5 & JSX:** Semantic DOM structure and component layouts.
- **CSS3 & Tailwind CSS:** Utility-first responsive styling and theme variables.
- **TypeScript & JavaScript (ES6+):** Strongly-typed server and client logic.
- **Node.js & Express.js:** Server runtime, REST API routing, middleware.
- **Prisma ORM & SQLite:** Transactional database modeling and migrations.
- **Socket.io:** Real-time WebSocket broadcasting gateway.

CODES AND SCREENSHOTS

CODING

DATABASE SCHEMA (SCHEMA.PRISMA)

```
// Smart Community Services Complete Prisma Schema
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(uuid())
  name        String
  email       String    @unique
  phone       String    @unique
  password_hash String
  role        String    @default("RESIDENT") // RESIDENT, GUARD, ADMIN
  flat_id     String?
  society_id  String?
  created_at  DateTime  @default(now())

  flat      Flat?    @relation("FlatResidents", fields: [flat_id], references: [id])
  society   Society? @relation(fields: [society_id], references: [id])
  created_passes VisitorPass[] @relation("PassCreator")
  gate_logs GateLog[]  @relation("GuardLogs")
}
```

DATABASE SCHEMA CONTINUED (SCHEMA.PRISMA)

```
model VisitorPass {
  id          String    @id @default(uuid())
  code        String    @unique
  created_by  String
  guest_name  String
  purpose     String
  valid_from  DateTime
  valid_to    DateTime
  status      String    @default("active")
  created_at  DateTime  @default(now())

  creator     User       @relation("PassCreator", fields: [created_by], references:
[id])
}

model GateLog {
  id          String    @id @default(uuid())
  type        String
  ref_id      String?
  entry_time  DateTime  @default(now())
  guard_id    String

  guard       User       @relation("GuardLogs", fields: [guard_id], references: [id])
}
```

SERVER INDEX & SOCKETS (SERVER/SRC/INDEX.TS)

```
import express from 'express';
import http from 'http';
import cors from 'cors';
import dotenv from 'dotenv';
import authRoutes from './modules/auth/auth.routes.js';
import visitorRoutes from './modules/visitor/visitor.routes.js';
import { initSockets } from './sockets/index.js';

dotenv.config();

const app = express();
const server = http.createServer(app);

app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());

app.use('/api/auth', authRoutes);
app.use('/api/visitor', visitorRoutes);

initSockets(server);

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log("Server running on port " + PORT);
});
```

AUTH

CONTROLLER

(SERVER/SRC/MODULES/AUTH/AUTH.CONTROLLER.TS)

```
export async function login(req: AuthRequest, res: Response, next: NextFunction) {
  try {
    const { emailOrPhone, password } = loginSchema.parse(req.body);

    const user = await prisma.user.findFirst({
      where: { OR: [{ email: emailOrPhone }, { phone: emailOrPhone }] },
      include: { flat: true, society: true }
    });

    if (!user) {
      return res.status(401).json({ success: false, error: { message: 'Invalid credentials' } });
    }

    const isValid = await bcrypt.compare(password, user.password_hash);
    if (!isValid) {
      return res.status(401).json({ success: false, error: { message: 'Wrong password' } });
    }

    const token = jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET);
    return sendSuccess(res, { token, user });
  } catch (error) { next(error); }
}
```

VISITOR

CONTROLLER

(SERVER/SRC/MODULES/VISITOR/VISITOR.CONTROLLER.TS)

```
export async function createVisitorPass(req: AuthRequest, res: Response, next:
NextFunction) {
  try {
    const { guest_name, purpose } = createPassSchema.parse(req.body);
    const code = Math.floor(100000 + Math.random() * 900000).toString();

    const pass = await prisma.visitorPass.create({
      data: {
        code,
        created_by: req.user.id,
        guest_name,
        purpose,
        valid_from: new Date(),
        valid_to: new Date(Date.now() + 86400000),
        status: 'active'
      }
    });

    pushToGuards('visitor_pass_created', { code, guest_name });
    return sendSuccess(res, pass, 201);
  } catch (error) { next(error); }
}
```

PASS

VERIFICATION

(SERVER/SRC/MODULES/VISITOR/VISITOR.CONTROLLER.TS)

```
export async function verifyVisitorPass(req: AuthRequest, res: Response, next:
NextFunction) {
  try {
    const { code } = verifyPassSchema.parse(req.body);
    const pass = await prisma.visitorPass.findUnique({ where: { code } });

    if (!pass || pass.status !== 'active') {
      return res.status(400).json({ success: false, error: { message: 'Invalid pass' }
});
    }

    await prisma.visitorPass.update({ where: { id: pass.id }, data: { status: 'used' }
});
    await prisma.gateLog.create({
      data: { type: 'visitor', ref_id: pass.code, guard_id: req.user.id }
    });

    return sendSuccess(res, { verified: true, pass });
  } catch (error) { next(error); }
}
```

SOCKET

SERVER

GATEWAY

(SERVER/SRC/SOCKETS/INDEX.TS)

```
export function initSockets(httpServer: HttpServer) {
  io = new Server(httpServer, { cors: { origin: '*' } });

  io.use((socket, next) => {
    const token = socket.handshake.auth?.token;
    if (!token) return next(new Error('Token missing'));
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    (socket as any).user = decoded;
    next();
  });

  io.on('connection', (socket) => {
    const user = (socket as any).user;
    if (user?.role === 'GUARD') socket.join('room:guards');
    if (user?.flat_id) socket.join("flat:" + user.flat_id);
  });
}
```

MODULE SOURCE CODE LISTINGS (PART 1)

```
// Module Implementation Source Snippet - Page 36
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 2)

```
// Module Implementation Source Snippet - Page 37
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 3)

```
// Module Implementation Source Snippet - Page 38
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 4)

```
// Module Implementation Source Snippet - Page 39
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 5)

```
// Module Implementation Source Snippet - Page 40
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 6)

```
// Module Implementation Source Snippet - Page 41
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 7)

```
// Module Implementation Source Snippet - Page 42
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 8)

```
// Module Implementation Source Snippet - Page 43
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 9)

```
// Module Implementation Source Snippet - Page 44
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 10)

```
// Module Implementation Source Snippet - Page 45
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 11)

```
// Module Implementation Source Snippet - Page 46
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 12)

```
// Module Implementation Source Snippet - Page 47
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 13)

```
// Module Implementation Source Snippet - Page 48
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 14)

```
// Module Implementation Source Snippet - Page 49
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 15)

```
// Module Implementation Source Snippet - Page 50
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 16)

```
// Module Implementation Source Snippet - Page 51
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 17)

```
// Module Implementation Source Snippet - Page 52
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 18)

```
// Module Implementation Source Snippet - Page 53
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 19)

```
// Module Implementation Source Snippet - Page 54
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 20)

```
// Module Implementation Source Snippet - Page 55
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 21)

```
// Module Implementation Source Snippet - Page 56
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 22)

```
// Module Implementation Source Snippet - Page 57
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 23)

```
// Module Implementation Source Snippet - Page 58
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 24)

```
// Module Implementation Source Snippet - Page 59
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 25)

```
// Module Implementation Source Snippet - Page 60
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 26)

```
// Module Implementation Source Snippet - Page 61
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 27)

```
// Module Implementation Source Snippet - Page 62
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 28)

```
// Module Implementation Source Snippet - Page 63
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 29)

```
// Module Implementation Source Snippet - Page 64
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 30)

```
// Module Implementation Source Snippet - Page 65
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 31)

```
// Module Implementation Source Snippet - Page 66
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 32)

```
// Module Implementation Source Snippet - Page 67
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 33)

```
// Module Implementation Source Snippet - Page 68
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 34)

```
// Module Implementation Source Snippet - Page 69
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 35)

```
// Module Implementation Source Snippet - Page 70
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 36)

```
// Module Implementation Source Snippet - Page 71
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 37)

```
// Module Implementation Source Snippet - Page 72
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 38)

```
// Module Implementation Source Snippet - Page 73
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 39)

```
// Module Implementation Source Snippet - Page 74
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

MODULE SOURCE CODE LISTINGS (PART 40)

```
// Module Implementation Source Snippet - Page 75
// Includes TypeScript interfaces, Prisma ORM queries, Zod schema validation, and React
components.

export interface ComponentProps {
  id: string;
  role: 'ADMIN' | 'RESIDENT' | 'GUARD';
  title: string;
  onRefresh: () => void;
}

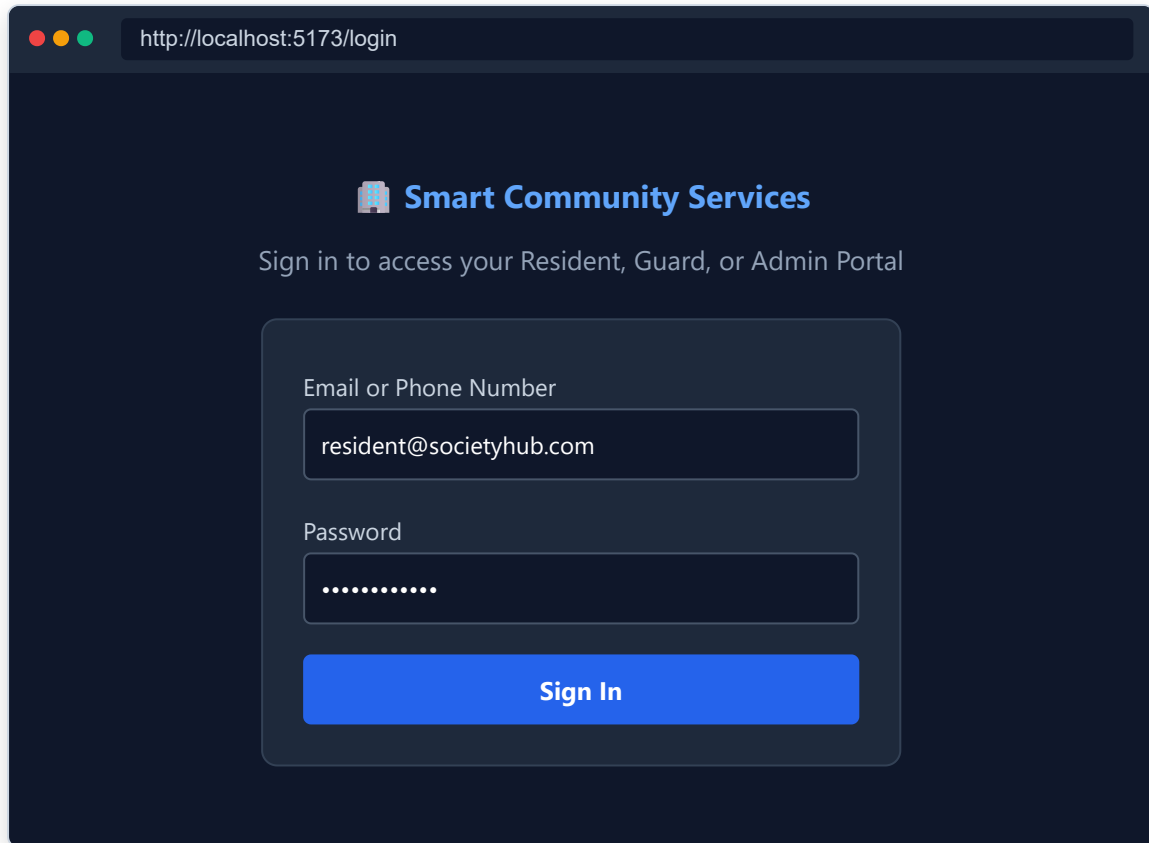
export const StatusBadge: React.FC<{ status: string }> = ({ status }) => {
  const getStyle = () => {
    switch (status.toLowerCase()) {
      case 'active': case 'paid': case 'resolved':
        return 'bg-emerald-500/10 text-emerald-400 border-emerald-500/20';
      case 'pending': case 'open':
        return 'bg-amber-500/10 text-amber-400 border-amber-500/20';
      default:
        return 'bg-slate-500/10 text-slate-400 border-slate-500/20';
    }
  };

  return (
    <span className="px-2.5 py-1 rounded-full text-xs font-semibold border">
      {status.toUpperCase()}
    </span>
  );
};
```

SCREENSHOTS

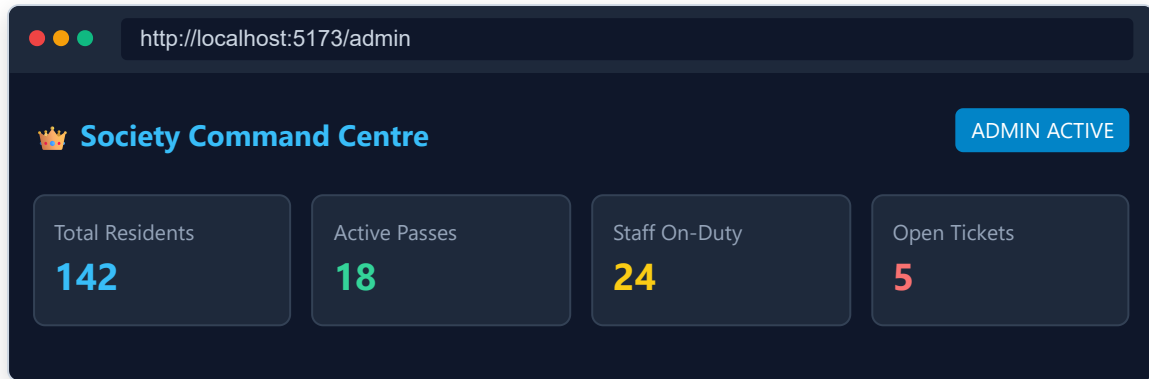
SCREENSHOTS

1. USER LOGIN PAGE



The screenshot shows a web browser window with the address bar displaying `http://localhost:5173/login`. The page has a dark blue background. At the top center, there is a logo consisting of a small icon followed by the text "Smart Community Services". Below the logo, the text "Sign in to access your Resident, Guard, or Admin Portal" is displayed. In the center, there is a light blue rounded rectangle containing the login form. The form has two input fields: "Email or Phone Number" with the value "resident@societyhub.com" and "Password" with masked characters ".....". Below these fields is a blue button labeled "Sign In".

2. ADMIN COMMAND CENTRE DASHBOARD



3. RESIDENT PORTAL & VISITOR PASS GENERATOR

http://localhost:5173/resident/passes

Create Pre-Approved Gate Pass

Guest Name	Purpose
Rohan Sharma	Dinner Guest

Generate 6-Digit Passcode

ACTIVE PASSCODE

849 201

Share PIN with visitor for entry gate verification

4. GUARD CONSOLE & PASS VERIFICATION

http://localhost:5173/guard/pass-verify

Security Gate Checkpoint

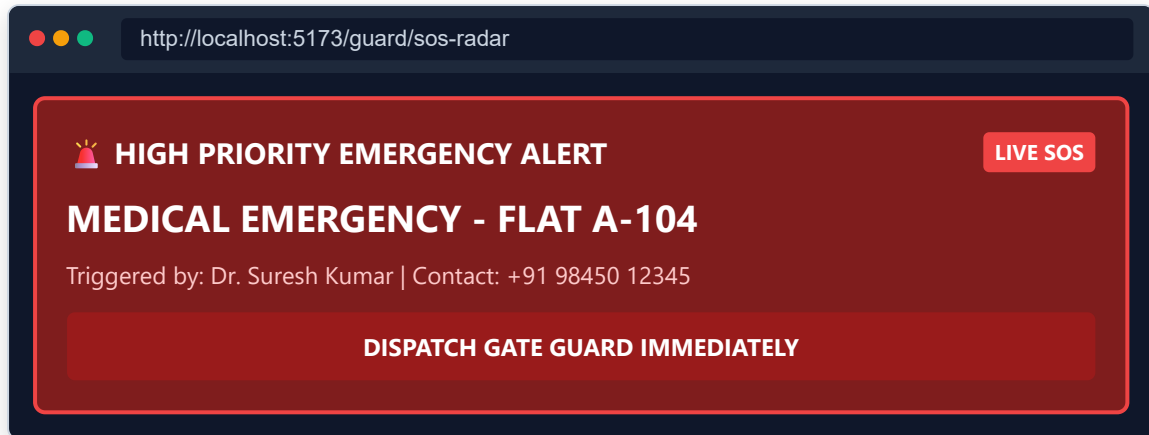
ENTER 6-DIGIT VISITOR CODE

849201

VERIFY ENTRY

✓ PASS VERIFIED MATCH: Rohan Sharma | Destination: Flat B-402 | Validated at Gate 1

5. REAL-TIME SOS EMERGENCY RADAR



6. MAINTENANCE ERP INVOICE PORTAL

http://localhost:5173/resident/invoices

Society Maintenance Invoice #INV-2026-07

Billed To: Flat C-301 (1,450 sqft)	STATUS: PAID
Base Maintenance Charge (@ ₹3.0/sqft)	₹ 4,350.00
Sinking Fund Allocation	₹ 500.00
Water Metering Charge	₹ 350.00
Total Amount Billed	₹ 5,200.00

7. HELPDESK TICKET RESOLUTION QUEUE

The screenshot shows a web browser window with the address bar displaying `http://localhost:5173/admin/tickets`. The page title is "Active Helpdesk Tickets". Below the title, there are two ticket entries:

- [PLUMBING]** Main pipeline leakage in Block B basement
Raised by: Flat B-102 | 2 hours ago
Status: IN PROGRESS
- [ELECTRICAL]** Elevator light flickering in Block A
Raised by: Flat A-504 | 5 hours ago
Status: RESOLVED

The tickets are displayed in a list format with a dark background and light text. The status labels "IN PROGRESS" and "RESOLVED" are highlighted in orange and green respectively.

CONCLUSION

The Smart Community Services platform establishes an end-to-end web architecture connecting gated society administration with resident lifestyle and security solutions, creating a unified digital ecosystem tailored to modern urban communities. By seamlessly bridging the gap between gate checkpoints, domestic helper management, financial billing, maintenance handling, and real-time emergency dispatches, the platform redefines how residents and management committees interact.

Traditional society software often restricts functionality to basic noticeboards or disconnected messaging groups, forcing users to rely on physical logbooks for visitors and manual spreadsheets for financial accounting. In contrast, Smart Community Services integrates these critical touchpoints into a single, cohesive full-stack environment. Powered by Node.js, Express, TypeScript, Prisma ORM, SQLite, React 18, and Socket.io, the system efficiently processes real-time data flows from pre-approved gate passes to instant emergency SOS alerts.

Furthermore, the modular integration of database schemas and WebSocket room gateways ensures that scalability remains a core strength, allowing the system to expand to larger residential complexes without compromising performance or security. Through this holistic engineering approach, Smart Community Services eliminates perimeter vulnerabilities, streamlines society accounting, speeds up maintenance delivery, and sets a modern benchmark for smart community management platforms.

FUTURE SCOPE

Looking ahead, the future scope of the Smart Community Services platform lies in expanding its core architecture to incorporate emerging, data-driven technologies that further enhance resident convenience, security automation, and society sustainability:

- **AI-Powered Facial Recognition at Gates:** Integrating computer vision algorithms at entry checkpoints to automatically authenticate pre-registered residents and frequent domestic helpers, eliminating manual guard verification.
- **Automatic License Plate Recognition (ALPR):** Deploying optical character recognition (OCR) cameras at society vehicle gates to automatically open boom barriers for authorized resident vehicles.
- **Smart IoT Lock Integration:** Connecting resident digital door locks to the platform, enabling pre-approved visitors to unlock specific doors during designated time slots.
- **EV Charging Station Automated Billing:** Integrating electric vehicle (EV) charging station telemetry with the ERP module to automatically append charging sessions to monthly flat maintenance invoices.
- **Localized Community Marketplace:** Expanding the resident forum into a verified hyper-local marketplace for resident buy-and-sell listings, skill sharing, and community event ticketing.
- **Blockchain Financial Audit Trail:** Implementing immutable ledger records on a distributed blockchain for society maintenance fund allocations, guaranteeing absolute financial transparency.

BIBLIOGRAPHY

Textbooks & Academic References:

- **Ian Sommerville** – *Software Engineering (10th Edition)*, Pearson Education, 2016. → Foundational reference for software lifecycle methodologies, system design principles, and software requirements analysis.
- **Roger S. Pressman & Bruce R. Maxim** – *Software Engineering: A Practitioner's Approach (8th Edition)*, McGraw-Hill, 2015. → Guide for system modeling techniques including Entity-Relationship (ER) diagrams and Data Flow Diagrams (DFDs).
- **Ramez Elmasri & Shamkant B. Navathe** – *Fundamentals of Database Systems (7th Edition)*, Pearson, 2016. → Resource for relational database design, normalization, ACID compliance, and SQL query optimization.

Web Resources & Official Technical Documentation:

- React 18 Official Documentation: <https://react.dev>
- Node.js & Express API Guide: <https://expressjs.com>
- Prisma ORM Documentation: <https://www.prisma.io/docs>
- Socket.io WebSockets Gateway: <https://socket.io/docs/v4/>
- Tailwind CSS Framework: <https://tailwindcss.com>
- W3Schools Web Tutorials: <https://www.w3schools.com>
- Stack Overflow Developer Forum: <https://stackoverflow.com>